

1. Differentiate List vs List Assignment Behavior (Interview Notes)

Why Can't List Be Assigned to List?

Although **Integer** is a subclass of **Number**, generics are **invariant** in Java.

This means:

```
List integers = new ArrayList<>();  
  
List numbers = integers;    // ❌ Compile-time Error
```

This is **not allowed**.

Why?

Suppose Java allowed it:

```
List integers = new ArrayList<>();  
  
List numbers = integers;  
  
numbers.add(3.14);    // Double
```

Now,

```
Integer x = integers.get(0);
```

The list would contain a Double, breaking type safety.

Therefore, Java **prevents the assignment at compile time**.

Correct Approach

Use wildcards.

```
List integers = new ArrayList<>();  
  
List numbers = integers;
```

Now you can safely read values as Number.

Important Interview Points

- Generics in Java are **invariant**.
 - Integer extends Number, but List does **not** extend List.
 - This restriction preserves **compile-time type safety**.
 - Use List when a method should accept lists of different numeric types.
 - A List is mainly for **reading**, not adding elements.
-

Tricky Interview Questions

Q1. Is List a subtype of List?

Answer: No.

Generics are **invariant**.

Q2. Why doesn't Java allow this assignment?

Answer:

To prevent inserting incompatible types and breaking type safety.

Q3. Which declaration accepts both List and List?

Answer:

```
List
```

Q4. Can we add a Double to List?

Answer: No.

The exact element type is unknown.

Interview Summary

- **List cannot be assigned to List because Java generics are invariant.**
- This prevents unsafe insertions and preserves type safety.
- Use **List** when flexibility is required.
- This is one of the most frequently asked Java Generics interview questions.

Interview One-Liner:

"Although Integer extends Number, List is not a subtype of List because Java generics are invariant."

2. Understand Type Erasure at a High Level (Interview Notes)

What is Type Erasure?

Type Erasure is the process where the Java compiler **removes generic type information during compilation**.

Generics exist only at **compile time**.

At runtime, the JVM sees only the raw types.

Example

Source code:

```
List list = new ArrayList<>();
```

After compilation (conceptually):

```
List list = new ArrayList();
```

The compiler inserts casts where necessary to maintain type safety.

Why Does Java Use Type Erasure?

To maintain **backward compatibility** with code written before Java 5, when generics did not exist.

Older libraries can still work with newer generic code.

Important Interview Points

- Generics are a **compile-time feature**.
 - Generic type information is **removed during compilation**.
 - The JVM does **not know** the actual generic type at runtime.
 - The compiler automatically inserts required casts.
 - Type erasure ensures **backward compatibility** with legacy Java code.
 - Because of type erasure, some operations (like creating new T() or checking instanceof List) are not allowed.
-

Tricky Interview Questions

Q1. What is type erasure?

Answer:

The compiler removes generic type information before bytecode is generated.

Q2. Are generics available at runtime?

Answer: No.

They are erased during compilation.

Q3. Why can't we write?

```
new T();
```

Answer:

Because the actual type of T is unknown at runtime due to type erasure.

Q4. Why is this illegal?

```
if (list instanceof List)
```

Answer:

Because generic type information does not exist at runtime.

Q5. Why did Java choose type erasure?

Answer:

To maintain backward compatibility with pre-Java 5 code.

Interview Summary

- **Type erasure removes generic type information during compilation.**
- Generics provide **compile-time type safety**, not runtime type information.
- The JVM works with raw types after compilation.
- Type erasure enables backward compatibility but introduces some generic limitations.

Interview One-Liner:

"Java generics use type erasure, meaning type parameters exist only at compile time and are removed before runtime."

3. Use Generics in Repository-Style Classes (Interview Notes)

What is a Repository-Style Class?

A **repository** is a class responsible for storing, retrieving, updating, and deleting objects.

Instead of creating separate repositories for each class, generics allow one reusable repository.

Without Generics

```
class StudentRepository {
    Student findById(int id) {
        return null;
    }
}

class ProductRepository {
    Product findById(int id) {
        return null;
    }
}
```

This results in duplicated code.

With Generics

```
class Repository {

    public void save(T obj) {
        System.out.println("Saved");
    }

    public T findById(int id) {
        return null;
    }
}
```

Using it:

```
Repository studentRepo = new Repository<>();  
  
Repository productRepo = new Repository<>();
```

The same repository works for different entity types.

Why Use Generic Repositories?

- Eliminates duplicate code.
- Improves reusability.
- Ensures compile-time type safety.
- Makes code easier to maintain.

This concept is widely used in **Spring Data JPA**.

Example:

```
interface JpaRepository {  
}
```

Here,

- User → Entity type
 - Long → Primary key type
-

Important Interview Points

- Generic repositories work with **any entity type**.
 - They eliminate repetitive CRUD code.
 - Type safety is enforced at compile time.
 - This pattern is commonly used in **Spring Boot**, **Spring Data JPA**, and many ORM frameworks.
 - JpaRepository is a real-world example of Java generics.
-

Tricky Interview Questions

Q1. Why use generics in repositories?

Answer:

To write one reusable repository instead of separate repositories for every entity.

Q2. What do the two generic parameters in JpaRepository represent?

Answer:

- User → Entity type
 - Long → Primary key type
-

Q3. What is the biggest advantage of a generic repository?

Answer:

Code reuse while maintaining compile-time type safety.

Q4. Where are generic repositories commonly used?

Answer:

- Spring Boot
 - Spring Data JPA
 - Hibernate
 - ORM-based applications
-

Interview Summary

- **Generic repositories provide a reusable way to perform CRUD operations on different entity types.**
- They reduce code duplication and improve maintainability.
- Compile-time type safety ensures only the correct entity type is used.
- This pattern is heavily used in enterprise Java frameworks like **Spring Data JPA**.

Interview One-Liner:

"Generic repositories use type parameters to create reusable, type-safe CRUD operations for multiple entity types, as seen in JpaRepository."